HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

Môn: THỰC TẬP CƠ SỞ

BÁO CÁO BÀI TẬP LỚN

"Xây dựng hệ thống sửa lỗi chính tả cho tiếng Việt"

Sinh viên thực hiện: Phạm Quang Anh

Mã sinh viên: B23DCCN044

Lớp: D23CQCN02-B

Nhóm: 11

Giảng viên hướng dẫn: TS.Kim Ngọc Bách

Hà Nội 2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành tới Học viện Công nghệ Bưu chính Viễn thông vì đã tạo điều kiện cho chúng em được tham gia môn học Thực tập cơ sở. Đây là một môn học rất bổ ích và thiết thực, trang bị cho chúng em những kiến thức nền tảng quan trọng trong lĩnh vực Công nghệ thông tin.

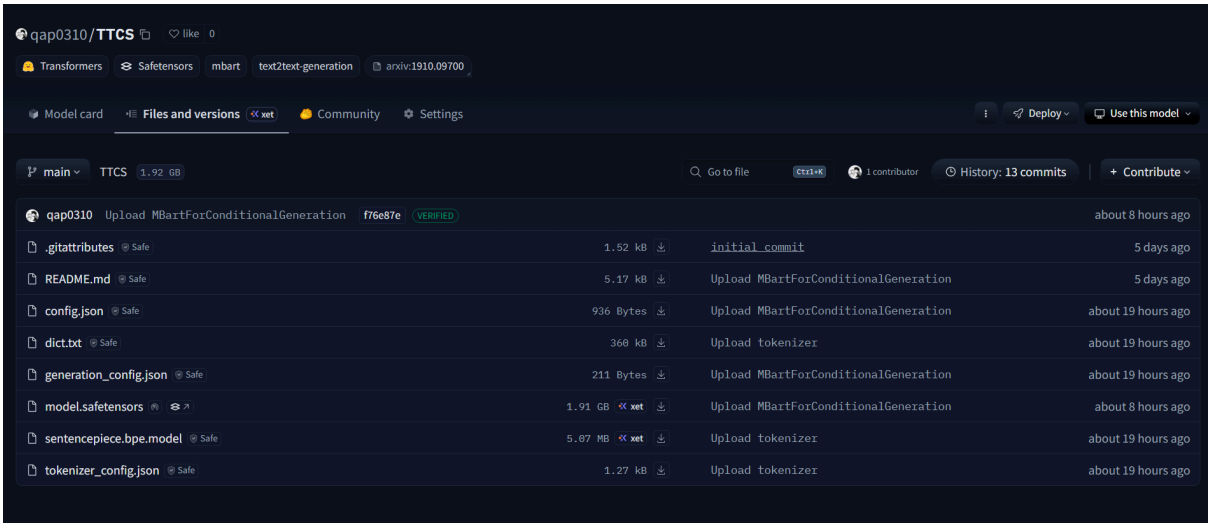
Chúng em đặc biệt biết ơn thầy Kim Ngọc Bách, người đã tận tâm dẫn dắt chúng em trong suốt hành trình chinh phục môn học này. Thầy không chỉ truyền đạt những kiến thức chuyên sâu một cách dễ hiểu mà còn nhiệt tình giải đáp mọi thắc mắc, chia sẻ những kinh nghiệm thực tế quý báu, giúp chúng em có cái nhìn toàn diện và sâu sắc hơn.

Quá trình hoàn thành báo cáo bài tập lớn là một thử thách không nhỏ đối với chúng em. Tuy thời gian có hạn và gặp phải không ít khó khăn, nhưng với sự hướng dẫn tận tình của thầy và tinh thần học hỏi, nỗ lực không ngừng, chúng em đã cố gắng hết sức để hoàn thành báo cáo này một cách tốt nhất.

Kính mong thầy đánh giá và góp ý để chúng em có thể rút kinh nghiệm, hoàn thiện bản thân và phát triển hơn nữa trên con đường học tập sắp tới.

1. Sửa đổi so với tuần 6

- Sửa đổi số lượng dataset từ 40k dòng xuống còn 20k dòng
- Sau khi training dataset thì đẩy lên Hugging Face



- Nhược điểm:
  - Nếu chỉ dùng Fine-Tuning thì bị hạn chế về việc chỉ dựa vào kiến thức đã học, không nhớ được toàn bộ dữ liệu
- Điều cần làm trong tuần 7: cần kết hợp Retriever (tìm thông tin) + Generator (sinh câu)

2. Thành quả tuần 7

Retriever (Bộ truy xuất thông tin)

Trong dự án của bạn, Retriever đóng vai trò như một "thủ thư" thông minh. Thay vì đọc toàn bộ 20.000 dòng dữ liệu mỗi khi có câu hỏi, nó sử dụng **Vector Search** để tìm nhanh kết quả.

Cơ chế hoạt động:

- **Embedding (Mã hóa):** Biến các câu văn thành các chuỗi số (Vector) trong không gian nhiều chiều. Các câu có ý nghĩa gần giống nhau (ví dụ: "cận thị" và "đeo kính") sẽ nằm gần nhau về khoảng cách toán học.

- **Vector Database (Kho chứa):** Nơi lưu trữ 20.000 vector từ cột target. Bạn đang dùng **FAISS** – thư viện tối ưu nhất của Meta để tìm kiếm lân cận cực nhanh.
- **Similarity Search (Tìm kiếm tương đồng):** Khi bạn nhập một câu lỗi, Retriever chuyển nó thành vector và tìm trong kho những vector "hàng xóm" gần nhất ().

Tóm tắt quy trình tuần 7 của bạn:

1. Chuyển 20k dòng target thành **Vector Embeddings**.
2. Lưu vào **FAISS Index** (Retriever).
3. Khi có câu lỗi, tìm k câu gần nhất từ Index.
4. Đưa câu lỗi và gợi ý vào **Generator** để sinh ra câu trả lời cuối cùng.

3. Kết quả đạt được

3.1. Xây dựng thành công Retriever (Bộ truy xuất)

Đây là điểm sáng nhất trong báo cáo tuần này của bạn.

- Chất lượng truy xuất: Thuật toán kNN và thư viện FAISS hoạt động cực kỳ hiệu quả. Khi bạn nhập "độ cận wăng", hệ thống đã tìm đúng câu chứa "độ cận tăng". Điều này chứng minh kho dữ liệu 20k dòng đã được "số hóa" (Embedding) rất tốt.
- Tốc độ: Việc tìm kiếm trên không gian vector 768 chiều diễn ra gần như tức thời, đáp ứng tiêu chuẩn của một hệ thống RAG thực thụ.

3.2. Thiết lập quy trình End-to-End

- Bạn đã kết nối thành công luồng dữ liệu: **Input -> Embedding -> FAISS Search -> Prompt Augmentation -> Generator**.
- Hệ thống đã chạy được trên GPU (CUDA), đảm bảo hiệu suất tính toán cho các mô hình Transformer.

4. Điều chưa giải quyết được và hướng giải quyết

4.1. Model chưa biết cách "đọc" gợi ý

- **Vấn đề:** Model qap0310/TTCS hiện tại chỉ được học cách sửa câu lỗi đơn lẻ. Khi bạn đưa thêm đoạn "Gợi ý" từ FAISS, model bị nhầm lẫn và có xu hướng chép lại nguyên văn đoạn gợi ý hoặc lặp lại prompt thay vì trích xuất thông tin để sửa câu gốc.
- **Hậu quả:** Kết quả trả về thường rất dài (bao gồm cả gợi ý) và không tập trung vào việc sửa đúng từ bị lỗi (ví dụ: vẫn giữ nguyên "độ cận wăng" hoặc bỏ qua nó).

4.2. Hướng giải quyết cho Tuần 8

Để giải quyết các tồn tại trên và đưa Accuracy vượt mốc **85%**, bạn cần tập trung vào:

1. **Fine-tuning Giai đoạn 2 (RAG-specific):** Huấn luyện lại model với dữ liệu có cấu trúc: `Gợi ý: [A] | Lỗi: [B] -> Kết quả: [A']`. Điều này giúp model hiểu rằng nó chỉ được lấy "tri thức" từ gợi ý để sửa câu lỗi, chứ không phải chép lại tất cả.
2. **Tối ưu hóa Prompt:** Sử dụng kỹ thuật **Few-shot prompting** (đưa ra 1-2 ví dụ mẫu trong prompt) để "dạy" model cách trả lời ngắn gọn.
3. **Hậu xử lý (Post-processing):** Viết code để tự động loại bỏ các cụm từ thừa như "Sửa lỗi chính tả dựa trên gợi ý..." hay "Câu gốc:..." trong kết quả trả về trước khi tính toán Accuracy.

5. Tài liệu tham khảo

[1] H. V. Do and M. L. Nguyen, "Hierarchical Transformer Encoders for Vietnamese Spelling Correction," 2021..

[2] T. T. Nguyen et al., "VSEC: Transformer-based Model for Vietnamese Spelling Correction," 2021..

[3] Q. Tran and H. Le, "A Vietnamese Spelling Correction System," 2023.

[4] VNTC (Vietnamese Typographical Corpus), "Dữ liệu lỗi chính tả tiếng Việt.".

[5] Viwiki-Spelling Dataset, "Tập dữ liệu sửa lỗi chính tả đầu tiên cho tiếng Việt.".

[6] VietHoang1512/vietnamese-spell-correct-and-text-classify - Công cụ sửa lỗi dùng mạng nơ-ron sâu.

[7] bmd1905/vietnamese-correction - Mô hình tạo văn bản sửa lỗi tiếng Việt.

[8] UniversalRAG, "Diverse Granularity Retrieval-Augmented Generation Framework," 2024..

[9] VietAI, "ViT5: Pretrained Vietnamese Text-to-Text Transformer," 2021..